

App:

Questa è la cartella principale dove viene gestita la logica dell'applicazione e dov'è contenuto maggior parte del codice. È divisa in tre sottocartelle chiave: **Models**, **Presenters**, e **Views**.

1. Models:

Models: La cartella **Models** contiene i modelli dell'applicazione, che corrispondono alle tabelle del database. In particolare:

- **BuildingModel.php:** Questo modello rappresenta gli edifici. Gestisce l'elenco di tutti gli edifici disponibili nel database, rendendo disponibile in gioco la selezione di edifici specifici in base al proprio ruolo, e la visualizzazione di quelli disponibili;
- **ClickModel.php:** Questo modello gestisce il click per votazione delle mappe. Specificatamente solo l'inserimento e la cancellazione del click;
- **DepartmentModel.php:** Si occupa della gestione della tabella pivot di Department. Infatti, al suo interno troviamo le funzioni per aggiornare i budget degli utenti;
- **MapBuildingModel.php:** Modello specifico per gestire le relazioni tra mappe e edifici. Tiene conto di tutti gli edifici piazzati nelle varie mappe, consentendo di trovare la posizione e l'edificio esatto per ciascuna. Infatti, viene usato per caricare, salvare e cancellare gli edifici piazzati;
- **MapModel.php:** Modello che rappresenta le mappe. Consente infatti di cancellare, creare, vedere, aggiornare una mappa;
- **MessageModel.php:** Si occupa della gestione dei messaggi. Infatti, consente di creare, cancellare e vedere tutti i messaggi riguardanti l'utente loggato;
- **RoleModel.php:** Rappresenta i ruoli disponibili nell'applicazione. Viene utilizzato per gestire la selezione e l'assegnazione di ruoli durante la creazione di una mappa e per il cambio di ruolo tramite messaggi.
- **UserModel.php:** Modello base per la gestione degli utenti. Gestisce la creazione di un nuovo utente, la sua autenticazione e visualizzazione dei dettagli degli utenti.

2. Presenters:

Questa cartella contiene i presenter, che elaborano i dati da mostrare nelle view seguendo il pattern MVP (Model-View-Presenter). Ogni presenter corrisponde a una tabella del database, rendendo il codice più modulare e facilitando l'elaborazione e validazione dei dati prima di passarli al rispettivo modello.

Ogni presenter, infatti, riceve nei parametri dei valori, che una volta elaborati e fatto i check di validazione, vengono mandati al proprio model nella funzione richiesta.

3. Views:

La cartella Views contiene i file necessari per la visualizzazione. Qui abbiamo tre sotto-parti:

- **CSS:** File di stile che gestiscono il layout e l'aspetto delle diverse pagine.
 - **createMap.css:** Gestisce lo stile della pagina per la creazione di una mappa;
 - **loader.css:** Gestisce la schermata di caricamento;
 - **loginPage.css:** Definisce lo stile per la pagina di login;
 - **map.css:** Contiene lo stile per la visualizzazione della mappa;
 - **mapChoice.css:** Stile del menu di scelta delle mappe;
 - **selectRole.css:** Stile per la schermata di selezione di un ruolo.
- **JS:** Script JavaScript per gestire l'interattività delle pagine. Questi file contengono codice per la gestione degli eventi, le chiamate API e altre logiche di frontend.
 - **createMap.js:** Si occupa della logica di creazione delle mappe. Contiene solo il codice per far funzionare il form di creazione della mappa, eseguendo la fetch di invio dei dettagli inseriti nella propria Route;
 - **loader.js:** Gestisce la durata della schermata di caricamento nelle varie pagine;
 - **map.js:** Logica JavaScript legata alla visualizzazione e interazione con la mappa. È il chunk di codice più ampio, in quanto gestisce tutte le logiche di visualizzazione dei dettagli dell'utente, degli edifici, dei messaggi, della mappa e tutte le logiche di inserimento/cancellazione degli edifici;
 - **mapChoice.js:** Gestisce la logica della pagina di scelta della mappa, come le anteprime delle mappe esistenti, gestendo la creazione dinamica delle div in base ai dati ricevuti dalla chiamata fetch alla propria Route. Contiene la gestione di altre logiche come la cancellazione automatica della mappa in caso di inattività, l'inserimento del click, la sua cancellazione, e la cancellazione della mappa al raggiungimento di una soglia di voti;
 - **selectRole.js:** Script per la gestione della schermata di selezione dei ruoli.
- **Pages:** File HTML che rappresentano le diverse pagine dell'applicazione.
 - **createMap.html:** La parte visiva della schermata di creazione di una mappa;
 - **map.html:** La struttura di una mappa prima del caricamento dei suoi dati;
 - **mapChoice.html:** Pagina dove l'utente può scegliere una mappa da una lista;
 - **registration.html:** Pagina per la registrazione di nuovi utenti;
 - **selectRole.html:** Pagina di selezione di un ruolo.

Config:

- **config.php:** Questo codice stabilisce una connessione a un database MySQL utilizzando PDO (PHP Data Objects, utilizzato per gestire l'accesso a Database indipendentemente dal tipo di Database). Utilizza le credenziali fornite (host, nome del database, utente e password) e, in caso di errore, cattura l'eccezione e mostra un messaggio di errore, garantendo così una gestione sicura della connessione.

Routes:

Questa cartella gestisce la definizione delle rotte dell'applicazione. Ogni file in Routes contiene la logica delle rotte specifiche per ciascuna entità, legata ai rispettivi Presenter, con un controllo dell'autenticazione basata sulle sessioni.

Ogni file di Route, gestisce la richiesta URI, suddividendola in parti e “leggendo” solo la parte utile per il backend: la funzione richiesta e gli eventuali dati passati.

Dopo la lettura, la validazione del metodo della richiesta (se GET, POST, UPDATE o DELETE), e trovata la rotta che la riguarda, passa al Presenter i dati ricevuti che verranno successivamente elaborati. Nel caso non venga trovata una rotta adatta (metodo indesiderato o funzione non trovata), ogni Route ha il proprio caso di default, che restituisce un messaggio di rotta non trovata, non ritornando quindi un risultato.

uploads:

Questa cartella è usata per archiviare file caricati dagli utenti, come le immagini di profilo o le immagini di copertina delle mappe. La validazione del formato dell'immagine ed il suo salvataggio, viene definita in ogni Presenter che le utilizza. Questa cartella è usata solo come contenitore per avere una cartella comune per tutte le immagini, avendo quindi un path predefinito senza dover ricercare ogni foto in posti separati.

docker-compose.yml:

Questo file è usato per orchestrare vari servizi Docker, come il database, l'applicazione web e altri servizi necessari al funzionamento del progetto. Definisce come ogni componente interagisce all'interno del container Docker. All'interno viene definito quale file leggere all'inizializzazione del DB, in modo da caricare i dati già esistenti (in questo caso il file init.sql).

Dockerfile:

Contiene le istruzioni per creare un'immagine Docker specifica per l'applicazione. Definisce quale versione di PHP usare, quali estensioni installare, come configurare il server web, e altro. Insieme al docker-compose.yml, è cruciale per lo sviluppo e il deployment in un ambiente containerizzato.

index.html:

È la homepage del progetto, la pagina di accesso. Non ha un file js annesso in quanto contiene un singolo form per l'autenticazione. Il form contiene la possibilità di inviare una chiamata POST ad un determinato URL senza per forza avere dello script di JavaScript.

init.sql:

Questo file contiene script SQL che inizializzano il database. Contiene la definizione delle tabelle, indici, chiavi esterne, e anche dati di esempio da popolare inizialmente. Viene eseguito quando si configura l'ambiente per la prima volta.